

COP290

Semester 2 2022-23

Lab 3

Learning Objectives: Reading code, designing and writing clean code, backend, frontend, CI/CD, SQL, testing.

Problem statement:

This lab will be done in groups of four. The main objective is to design and develop a website.

- You need to design and develop [APIs](#) required for a website and set-up a database on [MySQL](#). Data for your websites can be obtained online from websites like [Kaggle](#). Then, you will design the [frontend](#) for the website and get it running. You will also implement [unit](#) and integration tests and run them [continuously](#).
- Next step is to add a login page while the website is up and running, while checking if the other components are not getting affected using the tests implemented in the previous part. The login page should take basic information (with picture optional) from a user and then add the user to the database. It is advised to use an encryption algorithm to store passwords.
- Last step is to add an existing Machine Learning API as a feature of your website. You can choose an API hosted by google listed [here](#). This API can be used to make an existing feature better i.e. add recommendation to a selling website or have personalised feeds on Twitter type platform. Or you can plan and add a new feature altogether.

[Some website examples](#) are listed below. You can choose either one of these or any other website of your choice. The tasks should include building APIs, database, a login page along with multiple web pages.

1. Twitter / Stack Overflow / Reddit
 - a. Basic website with functionality to post opinions/questions, reply to other posts (comments/answers) and upvote or downvote the posts and comments/answers.
 - b. A post can be a combination of text and pictures.
 - c. Feature to follow other users to check out their posts and a feed for each user checking posts from the other users that they follow.
 - d. All the searched posts and the comments/answers for each post can be sorted on different orders: number of likes and time of posting.
 - e. User's profile page should display basic information about any registered user and their posts.

2. Bazar portal
 - a. Visitors should be able to see the items listed for sale
 - b. Registered users of the portal should have access to its primary features, which are listed below.
 - i. Buy and sell things
 - ii. Seller should be able to upload at-least product's name, price and 1 photograph.
 - iii. User's profile page that displays basic information about any registered user and the status of all the products pertaining to them.
 - c. You can build a buying system like Amazon or a bidding system like eBay.

Milestones and their end dates:

Week 1, 1st March

Problem statement and design idea documentation

Week 2, 8th March

Click-through prototype on figma

A frontend prototype will show different pages of your website and will show transitions after clicking different buttons.

This prototype will help you decide the page layout and different APIs you might need.

Using figma plugins, this prototype can give you a rough frontend starter code for all your webpages.

Database [ER Diagram](#)

Design the database in the form of an ER Diagram.

Have a look at <https://github.com/tmibvishal/healTrip> for an example ER Diagram.

Week 3, 15th March

APIs Design and Development

Design a structure of APIs in the form of a yaml file and visualise it with [Swagger](#).

You can use Swagger Codegen to generate starter code for a Python server and a Javascript client.

Connecting the backend (swagger generated code) with your database.

Week 4, 22nd March

Write unit and integration tests for your code and use github to run all the tests whenever you commit new code. Coverage (calculated by Python Library) should be greater than 60%.

Extended Week 5, 1st April

Building the frontend for all the basic features, integrating the backend with the frontend and getting the website up and running. Also, host the website on a Baadal VM.

Week 6, 8th April

Adding a login page feature and related web-pages. The task is to rely on unit and integration tests, so that the website doesn't fall apart.

Week 7, 15th April

Adding an existing ML Google API to the webpage without disturbing other components used in integration testing.

Week 8, 23rd April

Final touch ups and presentation!

Note that these are only recommended time allocation. Depending on the complexity and comfort with the language, different parts may take more or less time. It may be wiser to start on the next week's task as soon as you are done with the current week's tasks.

Submission instructions:

To track the project progress, we have some checkpoint submissions. No late extensions are provided.

Week 1, 1st March

Initial website design plan with the name of the website, team members, its overview, and expected features.

Deliverables: Pdf

Week 3, 15th March

Frontend prototype designed on figma. ER diagram for the database. API design and documentation on swagger.

Deliverables: Figma project link, a video of interacting with the frontend prototype, a pdf for ER diagram, and a yaml file for api design.

Week 5, 1st April

Up and running website

Deliverables: A short video of website features, github link to the code, test coverage report, and link of the website hosted on baadal VM

Week 8, 23rd April

Deliverables: You will be required to give a presentation and a demo of your website followed by Q & A. Also submit a final report with links to all the updated design and documentation (website documentation, figma link, ER diagram, swagger yaml file). Distribute 40 tokens among your team members specifying the workload split. Also specify who worked on what.

Grading:

Completeness: 10

Code cleanliness: 5

API design: 5

Frontend design: 5

Database design: 5

Login and ML API integration: 5

Final presentation and report (counted towards exam total): 15

Total: 50

References:

Web-development in Python

1. REST APIs: <https://realpython.com/api-integration-in-python/>
2. Flask & Frontend: <https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3>

MySQL:

1. <https://dev.mysql.com/doc/mysql-getting-started/en/>
2. <https://www.guru99.com/er-diagram-tutorial-dbms.html>

HTML/CSS/JavaScript:

<https://www.w3schools.com/>

Swagger for server and client starter code generation (<https://editor.swagger.io/>):

1. <https://www.youtube.com/watch?v=TybZad1zwes>

Figma (www.figma.com/):

1. <https://designcourse.com/blog/post/figma-tutorial---an-introduction-to-a-very-impressive-ui-design--prototyping-tool>
2. <https://www.animaapp.com/blog/design-to-code/how-to-export-figma-to-html/>

Continuous Integration / Continuous Delivery / Testing:

1. <https://www.infoworld.com/article/3271126/what-is-cicd-continuous-integration-and-continuous-delivery-explained.html>
2. <https://realpython.com/python-continuous-integration/>
3. <https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python>
4. Coverage:
<https://www.pythontutorial.net/python-unit-testing/python-unittest-coverage/>

API testing using Postman:

<https://learning.postman.com/docs/getting-started/introduction/>

UX design:

<https://www.nngroup.com/articles/#videos>

<http://baddesigns.com/examples.html>

Recommended books:

1. For designing user interfaces: The Design of Everyday Things by Don Norman.
2. For object oriented programming: Head First Design Patterns by Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra